

Overdetermined Systems - Run Instructions

Step-by-step instructions for reproducing the large overdetermined-system experiments, running the numerical benchmark on the University of Manchester CSF with Slurm, and regenerating the CSV and plot outputs.

Purpose of this document

These instructions describe the practical workflow used for the large overdetermined-system part of the dissertation. They are written for a reader who may have little or no previous experience with Python, Linux command-line work, or Slurm. The aim is to make the order of operations clear without assuming that the reader already knows how an HPC experiment is normally run.

The repository contains the numerical program `Overdetermined.py` and the post-processing program `Overdetermined_plots.py`. The numerical program does the expensive work: it constructs the synthetic sparse problems, runs the selected solver, records the measurements, and writes the numerical CSV files. The plotting program is run afterwards and uses those CSV files to recreate the figures and summary tables.

The numerical benchmark should be completed before the plotting stage is started. This separation is intentional. If the CSV results are retained separately, the plots can be regenerated without repeating the large matrix calculations.

Important: CSF access, account setup and the current University login procedure are not reproduced here. A valid CSF session is assumed. The current University instructions should be followed for obtaining and maintaining access.

1. Files in this folder

The overdetermined-system folder is deliberately kept small. The generated CSV and PNG files are outputs of the computation rather than permanent source files. The important files for reproduction are:

- Overdetermined.py - numerical benchmark and CSV generation.
- Overdetermined_plots.py - reads the benchmark CSV files and creates plots and summary CSV files.
- Overdetermined_Systems_Introduction.pdf - explains what the experiment is doing.
- Overdetermined_Systems_Requirements_and_Restrictions.pdf - records the requirements and restrictions that must be respected.
- Overdetermined_Systems_Run_Instructions.pdf - this document.

Generated results can be kept in a separate archive when an exact execution needs to be preserved. They do not have to be committed to the repository if the supplied programs can regenerate them.

2. Before starting

The large experiments were designed as an HPC workload rather than as a normal desktop or notebook calculation. The largest matrices, and particularly the Direct solver, can require substantial memory. The first step is therefore to make sure that the computing environment is appropriate before submitting any expensive job.

The following should be available before the benchmark is submitted:

- Valid University of Manchester CSF access.
- A compatible Python/Anaconda environment. The dissertation work used Anaconda 2024.10.
- The Python files from the repository, with their current filenames unchanged.
- A Slurm allocation that satisfies the current CSF rules.
- At least 128 GB of RAM for the Direct-solver cases. Allocations below this level are expected to fail for those cases.
- A wall-time allocation that is long enough for the selected experiment. The documented workflow used 24 hours.
- A CPU count that is permitted for the account and partition being used. CPU count is user-dependent and must be checked against current University rules.

3. Establish a CSF session

CSF login is an institutional procedure and is deliberately not specified by a fixed command in this repository. Login hosts, authentication arrangements and access procedures can change. The person reproducing the experiment should first follow the current University CSF guidance or training and establish a working terminal session on CSF.

Once the CSF shell is available, the remaining commands in this document can be entered in that shell. The large benchmark should not be run directly on the CSF login node. The login node is used for preparation, file management and submission; the numerical workload is submitted to Slurm.

4. Create a clean working directory

A separate working directory makes it easier to distinguish source files from generated results. The following is an example directory name; another directory may be used if preferred.

```
mkdir -p ~/overdetermined_systems
cd ~/overdetermined_systems
```

The two Python files should then be copied into this directory using the University's permitted repository or file-transfer method. After copying, check that the expected files are present:

```
ls -lh
ls -l Overdetermin Overdetermined_plots.py
```

The second command should list both files. If either file is missing, the numerical run should not be submitted yet.

5. Check the Python environment

The dissertation work used Anaconda 2024.10. On the CSF system, the available software modules can change. The environment should therefore be checked before running the benchmark.

The recorded workflow used the following module preparation:

```
module purge
module load apps/binapps/anaconda3/2024.10
```

If the current CSF installation uses a different module path or environment, the current University documentation takes precedence. The important point is that the Python interpreter used to execute the benchmark has the required scientific stack.

Check the interpreter and packages with:

```
python --version
python -c "import numpy, scipy, pandas, matplotlib; print('Scientific Python stack available')"
python Overdetermined.py
```

The final command is particularly useful because `Overdetermined.py` exposes the valid experiment and solver names through its command-line interface. It should display the experiment choices and the solver choices rather than beginning a benchmark.

6. How Overdetermined.py is called

Overdetermined.py is not intended to be started with only **python** Overdetermined.py for the Slurm benchmark. The current program expects the experiment and solver to be supplied as command-line arguments. The two required arguments are:

- --experiment - selects one of the six parameter studies.
- --solver - selects Direct, CG, LSQR or LSMR.

The six experiment names accepted by the program are:

Condition Number
Sparsity
Matrix Size
Lambda
Sparsity Pattern
Spectrum

The four solver names are:

Direct
CG
LSQR
LSMR

The other command-line options have defaults matching the supplied program:

- --repeats defaults to 10.
- --noise-level defaults to 0.01.
- --random-seed defaults to 42.

For example, one experiment/solver combination can be run with:

```
python Overdetermined.py
--experiment "Condition Number" \
--solver CG \
--repeats 10 \
--noise-level 0.01 \
--random-seed 42
```

This command is useful for understanding the interface, but a complete reproduction of the large benchmark should preserve the recorded multi-task organisation rather than treating one combination as the entire experiment.

7. The 24-element Slurm array

The large benchmark was organised as a 24-element Slurm array. This corresponds to six parameter studies multiplied by four solver methods. Each array element can therefore be assigned one experiment/solver combination. No historical Slurm job number is part of the experiment and none is required for reproduction.

The following mapping provides a clear way of preserving that organisation. The order is six studies, with Direct, CG, LSQR and LSMR for each study:

Array task	Experiment	Solver
1	Condition Number	Direct
2	Condition Number	CG
3	Condition Number	LSQR
4	Condition Number	LSMR
5	Sparsity	Direct
6	Sparsity	CG
7	Sparsity	LSQR

Array task	Experiment	Solver
8	Sparsity	LSMR
9	Matrix Size	Direct
10	Matrix Size	CG
11	Matrix Size	LSQR
12	Matrix Size	LSMR
13	Lambda	Direct
14	Lambda	CG
15	Lambda	LSQR
16	Lambda	LSMR
17	Sparsity Pattern	Direct
18	Sparsity Pattern	CG
19	Sparsity Pattern	LSQR
20	Sparsity Pattern	LSMR
21	Spectrum	Direct
22	Spectrum	CG
23	Spectrum	LSQR
24	Spectrum	LSMR

8. Prepare the Slurm submission script

The submission script below is a template for the recorded workflow. It deliberately leaves the CPU count as a placeholder because the number of CPUs available to a particular user depends on the current University allocation. A partition, account directive or other CSF-specific option may also be required.

```
#!/bin/bash
#SBATCH --job-name=overdetermined_benchmark
#SBATCH --time=24:00:00
#SBATCH --mem=128G
#SBATCH --cpus-per-task=<CPU_COUNT>
#SBATCH --array=1-24

module purge
module load apps/binapps/anaconda3/2024.10

cd ~/overdetermined_systems

case "$SLURM_ARRAY_TASK_ID" in
  1) EXP="Condition Number"; SOLVER="Direct" ;;
  2) EXP="Condition Number"; SOLVER="CG" ;;
  3) EXP="Condition Number"; SOLVER="LSQR" ;;
  4) EXP="Condition Number"; SOLVER="LSMR" ;;
  5) EXP="Sparsity"; SOLVER="Direct" ;;
  6) EXP="Sparsity"; SOLVER="CG" ;;
  7) EXP="Sparsity"; SOLVER="LSQR" ;;
  8) EXP="Sparsity"; SOLVER="LSMR" ;;
  9) EXP="Matrix Size"; SOLVER="Direct" ;;
  10) EXP="Matrix Size"; SOLVER="CG" ;;
  11) EXP="Matrix Size"; SOLVER="LSQR" ;;
  12) EXP="Matrix Size"; SOLVER="LSMR" ;;
  13) EXP="Lambda"; SOLVER="Direct" ;;
  14) EXP="Lambda"; SOLVER="CG" ;;
  15) EXP="Lambda"; SOLVER="LSQR" ;;
  16) EXP="Lambda"; SOLVER="LSMR" ;;
  17) EXP="Sparsity Pattern"; SOLVER="Direct" ;;
  18) EXP="Sparsity Pattern"; SOLVER="CG" ;;
  19) EXP="Sparsity Pattern"; SOLVER="LSQR" ;;
  20) EXP="Sparsity Pattern"; SOLVER="LSMR" ;;
  21) EXP="Spectrum"; SOLVER="Direct" ;;
  22) EXP="Spectrum"; SOLVER="CG" ;;
  23) EXP="Spectrum"; SOLVER="LSQR" ;;
  24) EXP="Spectrum"; SOLVER="LSMR" ;;
  *) echo "Invalid array task"; exit 1 ;;
esac
```

```
python Overdetermined.py experiment "$EXP" --solver "$SOLVER" --repeats 10 --noise-level 0.01 --random-seed 42
```

Save the script with a name such as `run_overdetermined.slurm`. The CPU placeholder must be replaced by a value actually permitted by the current CSF allocation. The current University documentation should also be checked for the appropriate partition and account settings before submission.

The 128 GB memory request is important for the Direct cases. Allocations below 128 GB are expected to fail for those cases. The 24-hour setting reflects the documented dissertation workflow, but the currently permitted wall time must be checked before submission.

9. Submit the job

From the directory containing the Python files and the Slurm script, submit the array with:

```
sbatch run_overdetermined.slurm
```

Slurm will return a new job identifier. That identifier is only a label for the current execution and is not part of the benchmark definition.

10. Monitor the Slurm array

The queue can be checked with:

```
squeue -u $USER
```

The important thing is to confirm that the array tasks complete successfully. A task that stops because of an out-of-memory condition, a time limit, a missing package or another execution error should not be treated as a completed benchmark.

The Slurm output files should be inspected if a task fails. The exact output-file names depend on the submission script and current Slurm behaviour. Commands such as the following can be used to inspect files in the working directory:

```
ls -lh  
find . -maxdepth 2 -type f | sort
```

11. Confirm the numerical CSV files

Overdetermined writes a checkpoint CSV during the numerical calculation and a final file named `results_final.csv` for each experiment/solver directory. The directory is formed from the output directory, experiment name and solver name.

The program uses the environment variable `ERP_OUTPUT_DIR` when one is set; otherwise its default output directory is `output_test`. This means that the actual location should be checked rather than assuming that an older execution used the same path.

```
find . -type f -name "results_final.csv" -print  
find . -type f -name "results.csv" -print
```

A successful numerical stage should produce final CSV files for the completed experiment/solver combinations. The CSV data contain the parameter information and measured quantities used in the dissertation analysis.

12. What is in the benchmark CSV?

The numerical output records the configuration of each run together with the measurements. The main fields include the experiment name, experiment number, run number, matrix dimensions, condition parameter, sparsity, sparsity pattern, spectrum, lambda, solver, runtime, iteration count, residual and relative error.

This is useful when checking a reproduction because the CSV is not just a list of times. It records the configuration under which each time was obtained.

13. Run the plotting stage

The plotting stage should only be started after the numerical CSV files have been checked. The plotting program searches the benchmark output for the completed result files, combines them, calculates summary tables and writes the requested figures.

Run it from the directory containing the plotting program, using the same compatible Python environment:

```
python Overdetermined_plots.py
```

The plotting script should be allowed to determine the output structure from its own current source code. This is preferable to manually recreating filenames from an old run, because the plotting code is the authoritative definition of the current post-processing stage.

14. Find the generated plots and summary CSV files

After the plotting program finishes, inspect the output tree. A useful general check is:

```
find . -type f \( -name "*.png" -o -name "*.csv" \) -print | sort
```

The PNG files are the regenerated figures. The summary CSV files are the post-processed numerical summaries. The original results_final.csv files contain the underlying benchmark observations.

The plotting code creates a dedicated plots directory under the benchmark output directory. It also creates summary CSV files for the overall results and for the individual study dimensions, including condition parameter, sparsity, matrix size, lambda, spectrum and sparsity pattern.

15. Copy the results to a local computer

If the CSV files or PNG figures need to be inspected on a local computer, they can be transferred from CSF using the University's permitted file-transfer method. The repository itself does not need to contain these generated files.

For a permanent record of a particular dissertation reproduction, the useful archive is the generated CSV/PNG output together with the repository version, execution date, Python and package versions, CSF partition, memory allocation, CPU allocation and wall-time allocation. This provides an execution record without making the generated files a permanent dependency of the source repository.

16. If a Direct-solver task fails

- Check the Slurm output for an out-of-memory message.
- Confirm that at least 128 GB of RAM was requested and that the request was actually granted.
- Check the current CSF memory limits and partition restrictions.
- Check whether the task reached the wall-time limit. The documented workflow used 24 hours.

A failure caused by insufficient memory should not be interpreted as evidence that the mathematical experiment is invalid. The Direct solver explicitly forms the normal-equation matrix and then performs the corresponding direct solution, so its memory demand can be much larger than the original sparse matrix.

17. If an iterative-solver task is unusually slow

- Check the CPU allocation actually assigned to the job.
- Check the Python and numerical-library environment.
- Check current system load and queue conditions.
- Confirm that the benchmark parameters have not been changed.
- Record the resource allocation before comparing the timing with the dissertation.

Runtime is a measurement of the whole computational environment, not only of the Python source. Different CPU allocations, processors, library versions and system conditions can therefore change the measured time even when the mathematical experiment is unchanged.

18. If the plotting program fails

- Confirm that Overdetermined completed successfully.
- Confirm that at least one valid results_final.csv file exists.
- Check that the plotting program is being run from the intended working directory.
- Check that the plotting program is reading the same output directory that contains the numerical results.
- Check that the CSV files have not been renamed or manually edited.

The plotting stage cannot recreate missing numerical results. If the benchmark CSV files are incomplete, the numerical stage should be investigated first.

19. Recommended order of execution

For a complete reproduction, the following order avoids most common mistakes:

1. Obtain CSF access using the current University procedure.
2. Create a clean working directory.
3. Copy Overdetermined.py and Overdetermined_plots.py into the working directory.
4. Load or activate the compatible Anaconda/Python environment.
5. Run Overdetermined.py and confirm the available experiment and solver names.
6. Check the current CSF rules for memory, CPU count, wall time and partition.
7. Request at least 128 GB for Direct-solver cases.
8. Use the documented 24-hour allocation where permitted and appropriate.
9. Preserve the 24-element array organisation.
10. Submit the Slurm array.
11. Monitor the array until the tasks have completed.
12. Check the generated results_final.csv files.
13. Run Overdetermined_plots.py .
14. Check the generated PNG figures and summary CSV files.
15. Record the software environment and Slurm resource allocation if the results are to be compared with the dissertation timings.

20. What must not be changed while reproducing the dissertation

- Do not change the matrix dimensions, sparsity values, condition parameters or lambda values used by the supplied benchmark.
- Do not replace the matrix-generation procedure with a different one.
- Do not change the random-number configuration or repeat handling without documenting the change.
- Do not change solver tolerances, stopping criteria or maximum iterations when the intention is to reproduce the reported experiment.
- Do not replace the matrix-free CG implementation with an explicitly formed alternative if the supplied implementation is being reproduced.
- Do not reduce the problem size merely because a smaller job is easier to run and then describe it as the same benchmark.

Environmental settings are different. A current CSF requirement may require a different partition, CPU count, memory syntax, account directive or environment activation command. Such changes are acceptable when they are necessary to use the current system, provided the mathematical

experiment itself is kept fixed and the changed resource settings are recorded.

21. Final reproducibility check

A run should be considered a reproduction only when the numerical problem and the supplied implementation have been preserved. The fact that a modified version happens to run successfully is not enough.

Before treating the results as final, check the following:

- The correct repository version of `Overdetermined.py` was used.
- The correct repository version of `Overdetermined_plots.py` was used.
- The six parameter-study names and four solver names were preserved.
- The numerical benchmark was submitted through Slurm rather than run as an ordinary laptop calculation.
- The 24-element array organisation was preserved.
- The Direct-solver tasks had access to at least 128 GB of RAM.
- The wall-time request was sufficient; the documented workflow used 24 hours.
- The CPU count was selected according to current CSF rules and recorded.
- The benchmark CSV files were generated successfully before plotting.
- The PNG figures and summary CSV files were regenerated from those numerical results.

22. Important note about current CSF conditions

This document records the computational arrangement used for the dissertation, but it cannot freeze University infrastructure. CSF partitions, maximum wall times, CPU limits, memory limits, module names and login procedures may change. Current University documentation therefore takes precedence for those operational details.

The fixed part of the reproduction is the experiment itself: the supplied Python implementation, its benchmark parameters, its solver formulations, its numerical settings and its random-number handling. The computing-resource settings should be adapted only as necessary to satisfy the current CSF environment, and those adaptations should be recorded.

23. Repository output policy

The generated CSV and PNG files are deliberately treated as reproducible outputs. They may be removed from the repository after they have been inspected or archived separately. Keeping them out of the repository prevents a large collection of execution artefacts from being mistaken for required source material.

The intended repository workflow is therefore simple: the Python programs and the three explanatory PDFs remain in the folder; the large numerical outputs are created when the experiment is reproduced.

End of run instructions. For the explanation of the experiment itself, read `Overdetermined_Systems_Introduction.pdf`. For the conditions that must remain fixed and the resource restrictions, read `Overdetermined_Systems_Requirements_and_Restrictions.pdf`.